EN3160 – Assignment 2: Fitting and Alignment

Name: Weerasinghe J.A.H.R. **Index Number:** 230697V **GitHub Repository:** EN3160 Assignment 2

Question 1 – Blob Detection

Objective

The objective of this question is to detect sunflower centres and estimate their radii using the Laplacian of Gaussian (LoG) and scale-space extrema.

Question 1 – Blob Detection

Method

1. Yellow mask generation

Yellow regions are identified using the colour channels. Morphological closing helps seal gaps between petals.

```
b, g, r = cv.split(im.astype(np.float32) / 255)
yellow = np.minimum(r, g) - b
yellow_bin = yellow > 0.25
mask = yellow_bin.astype(np.uint8)
mask = cv.morphologyEx(
    mask, cv.MORPH_CLOSE, np.ones((5, 5), np.uint8)
)
```

2. Mask refinement

Small dark holes are filled to obtain more complete sunflower regions.

```
filled = binary_fill_holes(mask)
holes, n = label(filled & (mask == 0))
areas = np.bincount(holes.ravel())
small = areas < 900
small[0] = False
mask = (mask.astype(bool) | small[holes]).astype(np.float32)
```

3. Scale-normalized LoG

Gaussian smoothing and the Laplacian are applied at multiple scales to detect circular structures of different sizes.

```
sigmas = np.arange(4, 36, 1.5)
stack = []
```

```
for s in sigmas:
    blur = cv.GaussianBlur(mask, (0, 0), s)
    log = cv.Laplacian(blur, cv.CV_32F)
    stack.append(-(s**2) * log)

stack = np.array(stack)
```

4. Scale-space extrema

Local maxima are detected across spatial positions and scales, retaining responses above the threshold.

```
local_max = maximum_filter(stack, size=(3, 3, 3))
k, y, x = np.nonzero(
    (stack == local_max) & (stack > 0.3)
)
```

5. Candidate filtering

Overlapping detections are suppressed. Candidates are retained if more than 50% of the pixels in the surrounding ring are yellow.

6. Results

The detected circles are displayed, and the total number of detections and five largest circles are reported.

Results

Input Image



Detection Output



Discussion

The yellow mask helps reduce detections outside the sunflower regions. However, variations in flower size, overlapping petals and background objects may still cause missed or false detections. The selected sigma range also limits the sizes of circles that can be detected.

Question 2 – RANSAC Line and Circle Fitting

Method

1. Generate noisy data

Generate 100 points: 50 noisy points around a circle and 50 noisy points around a straight line.

```
N = 100
half_n = N // 2

# Circle points
r = 10
x0_gt, y0_gt = 2, 3
s = r / 16
t = np.random.uniform(0, 2*np.pi, half_n)
n = s * np.random.randn(half_n)
x = x0_gt + (r+n)*np.cos(t)
y = y0_gt + (r+n)*np.sin(t)
X_circ = np.column_stack((x, y))

# Line points
s = 1.0
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m*x + b + s*np.random.randn(half_n)
X_line = np.column_stack((x, y))

X = np.vstack((X_circ, X_line))
```

2. Set RANSAC parameters

- $p = 0.99$: Target probability of selecting at least one all-inlier sample.
- $e = 0.5$: Assume 50% of the points are outliers when estimating the iteration count.
- $T_{\text{line}} = 1.5$: Chosen based on the line noise standard deviation of 1.
- $T_{\text{circle}} = 1.5(10/16) = 0.9375$: Chosen based on the circle's radial noise standard deviation of $r/16 = 10/16$.

The minimum sample sizes are 2 points for a line and 3 points for a circle.

```
p = 0.99
e = 0.5

s_line = 2
s_circ = 3
```

```
N_line = int(np.log(1-p) /
             np.log(1-(1-e)**s_line))
N_circ = int(np.log(1-p) /
             np.log(1-(1-e)**s_circ))

T_line = 1.5
T_circ = 1.5 * 10/16
```

3. Fit the line using RANSAC

Randomly select two points to estimate a line. The general equation (ax+by=d) is obtained algebraically from the sampled points. Count inliers using the perpendicular distance threshold and retain the model with the most inliers. Refine the model using SVD on the inliers.

4. Fit the circle using RANSAC

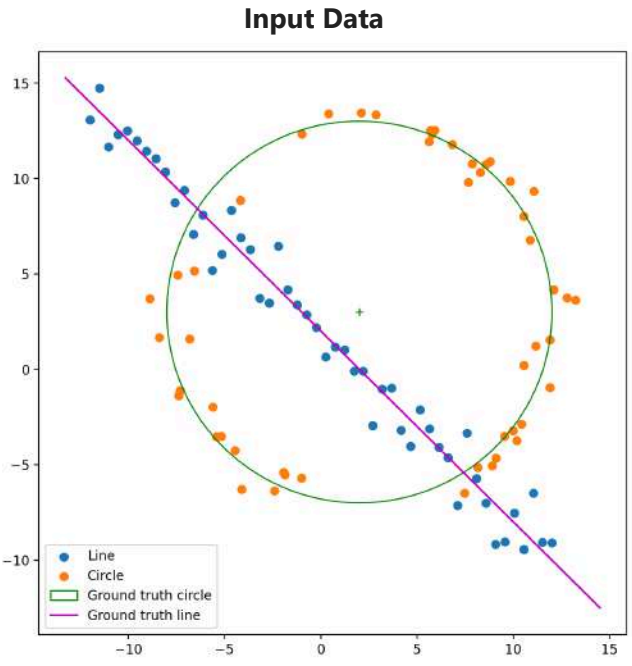
Randomly select three non-collinear points. The general circle equation (x^2+y^2+Ax+By+C=0) is solved algebraically to determine the centre and radius. Count inliers using the radial error threshold and retain the model with the most inliers.

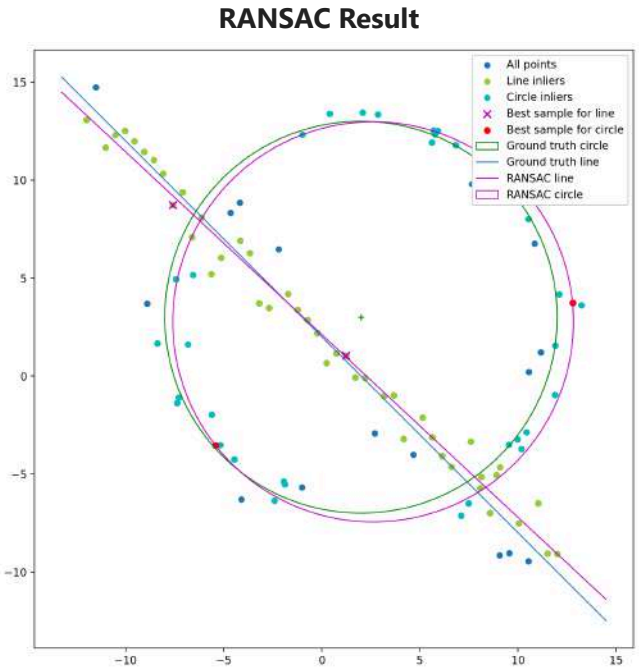
5. Sequential fitting

Fit the line first, remove its inliers from the dataset, and apply circle RANSAC to the remaining points.

6. Visualisation

Plot all points, line and circle inliers, selected samples, ground-truth models and RANSAC-fitted models to compare the results.





Discussion

RANSAC identifies the dominant line and circle despite noise and outliers. The fitted models can be compared with the ground-truth models to evaluate their accuracy. Removing the line inliers before circle fitting helps separate the two structures. Results may vary between runs because the samples are randomly selected.

Question 3 – Homography and Image Overlay

Method

1. Select the target surface

Load the scene image and select four corner points defining the planar surface where the second image will be overlaid.

2. Compute the homography

Map the four corners of the source image to the selected scene coordinates using a perspective transformation.

```
h, w = flag.shape[:2]

src = np.float32([
    [0, 0], [w-1, 0],
    [w-1, h-1], [0, h-1]
])

dst = np.float32([
    [110, 171], [507, 69],
    [730, 542], [275, 675]
])

H = cv2.getPerspectiveTransform(src, dst)
```

3. Warp and overlay the image

Apply the homography to warp the source image into the scene's perspective, then use a warped mask to blend it into the scene.

```
warped = cv2.warpPerspective(
    flag, H, (scene.shape[1], scene.shape[0])
)

warped_mask = cv2.warpPerspective(
    np.ones((h, w), dtype=np.uint8) * 255,
    H, (scene.shape[1], scene.shape[0])
)

result = scene.copy()
region = warped_mask > 0
result[region] = warped[region]
```

Results

Image Pair 1 - Falg

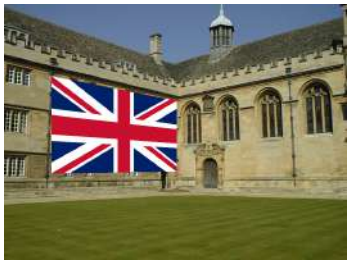
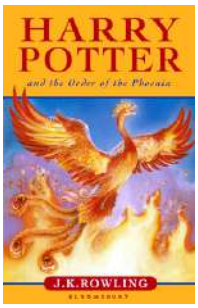
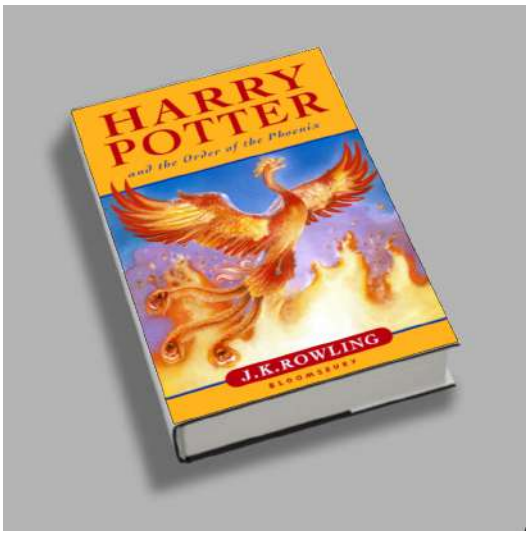
Input Scene	Input Image	Overlay Result
		

Image Pair 2 – Harry Potter

Input Scene	Input Image	Overlay Result
		

Discussion

- **Image Pair 1:** The flag is overlaid onto a building scene to demonstrate perspective transformation on a planar surface. The flag's corners are mapped to manually selected points on the building.
- **Image Pair 2:** The Harry Potter book cover is overlaid onto a blank book. This pair was chosen because the book cover provides a clear rectangular surface with visible perspective distortion.
- In both cases, the homography transforms the source image to match the perspective of the target surface. The warped mask ensures that only the transformed image region replaces the background.
- The alignment depends on accurate corner selection. Incorrect corner coordinates can cause distortion or misalignment.

Question 4 – PCB Alignment Using SIFT and RANSAC

Method

1. Feature detection and matching

SIFT detects distinctive features in the reference PCB (`AECC1.jpg`) and each target image. The ratio test filters ambiguous feature matches.

```
sift = cv.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

pairs = matcher.knnMatch(des1, des2, k=2)
good = [
    m for pair in pairs if len(pair) == 2
    for m, n in [pair]
    if m.distance < 0.65 * n.distance
]
```

2. Homography estimation

RANSAC estimates the homography from matched points while rejecting outliers. The target image is then warped to align with the reference PCB.

```
H, status = cv.findHomography(p2, p1, cv.RANSAC, 4.0)
aligned = cv.warpPerspective(image, H, (w, h))
```

3. Generate comparison images

Five outputs are saved for each successfully processed PCB image:

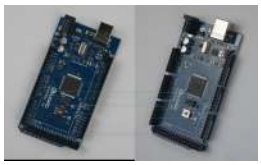
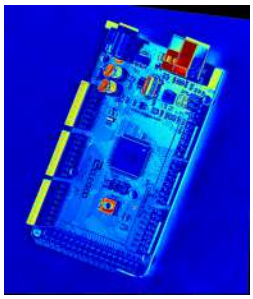
- **Feature matches:** Displays selected RANSAC inlier matches.
- **Aligned image:** Shows the perspective-corrected PCB.
- **Overlap image:** Blends the reference and aligned images to reveal misalignment.
- **Grayscale difference:** Shows the absolute pixel intensity differences.
- **Colour difference:** Applies the JET colour map to make differences easier to distinguish. Blue represents lower difference values, while warmer colours represent larger differences. These colours are artificial and do not represent the actual PCB colours.

The difference images are generated after Gaussian smoothing to reduce small variations before comparison. The colour-mapped differences are scaled using the 99.5th percentile of valid pixel differences to improve visibility.

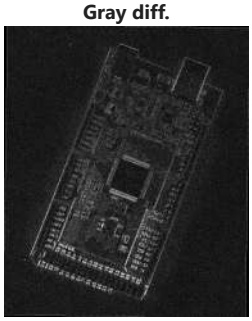
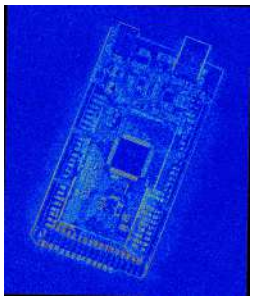
Discussion

SIFT and RANSAC help align PCB images despite changes in position and perspective. Overlap and difference images reveal alignment errors and visual differences between boards. Remaining differences may result from imperfect alignment, illumination changes, or actual component and board variations.

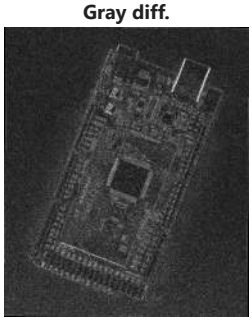
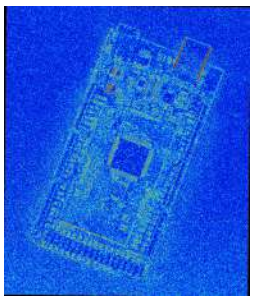
ADAD1

Matches	Aligned	Overlap	Gray diff.	Colour diff.
				

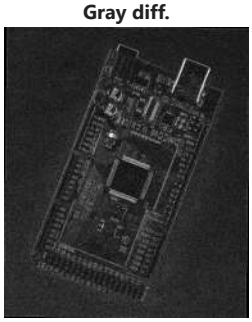
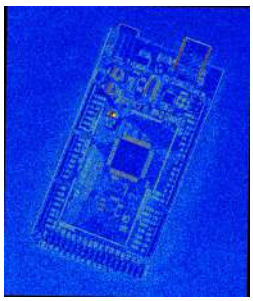
AECC2

Matches	Aligned	Overlap	Gray diff.	Colour diff.
				

AECC3

Matches	Aligned	Overlap	Gray diff.	Colour diff.
				

AECC4

Matches	Aligned	Overlap	Gray diff.	Colour diff.
				

Discussion and Comparison

- **Feature matching:** SIFT identified corresponding features between the reference PCB (`AECC1`) and the other PCB images. RANSAC rejected inconsistent matches and estimated the homography.

- **Alignment:** The aligned images show that the PCBs can be brought into a similar position and perspective. The overlap images make remaining misalignment easier to notice.
- **Grayscale difference:** The difference images highlight areas that do not match after alignment. Bright regions indicate larger intensity differences.
- **Colour difference:** The blue background mainly represents lower difference values, while brighter colours highlight larger differences. These colours are generated by the JET colour map and do not represent actual PCB colours.
- **Comparison between boards:** The results differ between PCB images. Some align closely with the reference, while others show more visible differences around components and board edges.
- **Possible causes:** Remaining differences may come from imperfect alignment, lighting and shadow variations, or actual differences in component positions and shapes.
- **Overall:** SIFT feature matching combined with RANSAC provides a practical way to align PCB images and identify regions that differ from the reference. However, the difference images alone cannot determine whether a difference is caused by a real hardware change or by image conditions.